# Final Project - Topic B: Dataset Forensics and Label Audit

## Dataset Forensics and Label Audit

*Audit a noisy classification dataset, identify trustworthy and untrustworthy examples, and explain how data quality changes the meaning of model evaluation.*

### I. Background

Many machine-learning failures are data failures. A classifier can look strong because near-duplicate examples leak across splits, shallow text artifacts expose labels, ambiguous examples make the task ill-defined, or a few mislabeled rows distort the error analysis. Improving the model is not enough if the dataset itself is not trustworthy.

In this project, you act as a dataset auditor. You will inspect a CPU-friendly text classification dataset built from the public UCI SMS Spam Collection, locate suspicious examples, decide which findings are real, and explain how different data-quality issues affect evaluation. The central challenge is judgment: not every suspicious row is wrong, and not every weak signal should become a high-confidence finding.

### II. Project Overview

The starter package includes:

- public data-source information for the UCI SMS Spam Collection;
- `data/split_manifest.csv`, which maps public raw rows to stable course ids and splits;
- `data/data_dictionary.md` describing the canonical working table;
- `examples/suspicious_examples_sample.csv`, which shows the audit submission format;
- `configs/audit_protocol.json`, which defines issue labels, allowed values, and public strict-check rules.

You must download the public SMS messages and labels yourself, align them with the split manifest, and write your own profiling, baseline, and audit code. The data is small enough for local pandas and scikit-learn work, but large enough that a manual row-by-row inspection is not a complete solution.

Final grading may use a hidden audit key containing known duplicate, leakage, label-error, shortcut, ambiguous, and false-positive-trap cases. The hidden key is used to reward precision, recall, evidence quality, and uncertainty calibration.

### III. Audit Targets

You must investigate six audit targets. For each target, produce reproducible evidence rather than only a claim.

| # | Audit target | Required question |
|---|---|---|
| 1 | Exact duplicate clusters | Which examples are exact duplicates, and do any duplicate clusters contain label or split conflicts? |
| 2 | Near-duplicate clusters | Which examples are near duplicates under a justified threshold, and which candidates should be rejected as false positives? |

| # | Audit target | Required question |
|---|---|---|
| 3 | Cross-split leakage | Are held-out examples recoverable from training examples or corpus artifacts? |
| 4 | Likely label errors | Which labels are suspicious under at least two independent signals? |
| 5 | Shortcut features | Can shallow features such as length, digits, contact tokens, promotional phrases, or row-order artifacts explain too much performance? |
| 6 | Ambiguous examples | Which examples admit multiple defensible labels, and how do they limit the score ceiling? |

## IV. Expected Evidence

Your audit should be ranked and evidence-backed. The goal is not to flag as many rows as possible; it is to separate strong findings, uncertain findings, and false positives with clear evidence.

You should design analyses that combine automated signals with manual judgment. Strong evidence may come from duplicate rules, nearest-neighbor agreement, model disagreement, shallow-feature baselines, or annotation-policy reasoning. The starter README specifies the exact `suspicious_examples.csv` schema.

High-confidence rows should have strong support. Bulk over-reporting is penalized: a long list of weak high-confidence findings is worse than a shorter ranked list that clearly explains uncertainty and rejected candidates.

## V. Analysis Scope

Your analysis should cover all six audit targets and explain how data quality affects evaluation. You should also compare at least two data interventions, such as relabel overlays, deduplication-by-cluster, removing suspicious training rows, or rebuilding the split by duplicate cluster.

Do not edit held-out labels. If you test a training-label correction, represent it as an experimental overlay file instead of overwriting the public labels. Details of required audit files are in the starter README.

## VI. Starter Package

The starter package provides the public data source, split manifest, audit protocol, and sample submission file. Use its `README.md` for the canonical row-id mapping, exact CSV schemas, and issue-label choices. You are responsible for writing the code that downloads, profiles, models, and audits the data.

## VII. Report Requirements

The final `report.pdf` should summarize your methodology, analysis, key findings, and conclusions. It should be self-contained and written as a coherent project report rather than a collection of audit outputs.

A strong report should make clear how you moved from raw data to ranked audit findings, why the findings are trustworthy, and where uncertainty remains. For this topic, the report should demonstrate judgment: strong evidence for true issues, explicit treatment of false positives, and a reasoned discussion of how data-quality problems affect evaluation.

The report should also contain a dedicated **Difficulties and Solutions** section with at least three concrete, verifiable challenges you encountered during the project and how you addressed them. Include an AI usage

declaration explaining how AI tools were used and how their outputs were verified.

## VIII. Deliverables

```
repo/
|-- audit/
|   |-- profile.md
|   |-- duplicates.md
|   |-- leakage.md
|   |-- label_noise.md
|   |-- shortcut_features.md
|   `-- ambiguity.md
|-- scripts/
|-- results/
|-- suspicious_examples.csv
|-- adjudication_memo.csv
|-- impact_analysis.md
|-- logs/chat.md
|-- report.pdf
`-- README.md
```

The `README.md` must contain exact commands and software versions needed to regenerate the main artifacts.

## IX. Academic Integrity and AI Usage

Using AI tools is expected, but the final audit must be your own evidence-backed work. You must cite datasets, code, tools, and any public analyses you read. Fabricated manual judgments or invented examples are academic-integrity violations.